

1. PROJECT OVERVIEW

- **Project Title:** ComicCraft - AI Comic Story Creator using Gemini Models

2. INTRODUCTION TO PROJECT PLANNING

The Project Planning phase is a critical step in the software development lifecycle of ComicCraft. It defines the roadmap, development milestones, resource allocation, and risk management strategies required to successfully build and deploy the AI-driven comic creator. Given the complexity of integrating multiple Generative AI models (Google Gemini and Hugging Face Stable Diffusion) with a FastAPI backend, a structured, milestone-based approach ensures systematic development, testing, and deployment.

3. AGILE METHODOLOGY & SPRINT PLANNING :

The development of ComicCraft follows an Agile methodology, breaking the project down into iterative sprints. This approach allows for continuous testing of AI APIs, immediate error resolution, and seamless integration of frontend UI with backend logic. The project is divided into five primary milestones, each focusing on a specific functional domain of the application.

4. MILESTONE BREAKDOWN:

Milestone 1: Model Selection and Architecture

- **Objective:** To establish the foundational architecture and select the appropriate AI models.
- **Activity 1.1 - AI Model Research:** Research capabilities of Google Gemini Flash (for structured JSON outlines), Gemini Pro (for detailed narrative creation), and Hugging Face Stable Diffusion (runwayml/stable-diffusion-v1-5) for image generation.
- **Activity 1.2 - Architecture Definition:** Define the 3-tier architecture: Frontend (HTML/CSS/Jinja2), Backend (FastAPI), and AI Integration (Gemini + Diffusers).

- **Activity 1.3 - Environment Setup:** Create a Python virtual environment and install necessary dependencies (fastapi, uvicorn, google-generativeai, diffusers, fpdf, pillow).

Milestone 2: Core Functionalities Development

- **Objective:** To build the standalone Python modules that communicate with the AI models and format the outputs.
- **Activity 2.1 - AI Scripts:** Develop `gemini_flash.py` to generate the 5-panel structured outline. Develop `gemini_pro.py` to expand the outline into a full comic story with dialogue. Develop `image_generator.py` to query the Stable Diffusion API and save panels.
- **Activity 2.2 - Layout & Export:** Develop `layout_builder.py` to structure the generated text and images. Implement `exporters.py` using the FPDF library to compile the complete comic layout into a downloadable multi-page PDF.

Milestone 3: routes.py Development (FastAPI)

- **Objective:** To set up the primary FastAPI routes that handle user requests and trigger the AI generation pipeline.
- **Activity 3.1 - Route Implementation:** Define core routes including `/` for the homepage, `/generate` to handle form submissions, and `/export` - success for download confirmation. Ensure the `/generate` route correctly invokes the outline, story, image, layout, and PDF functions sequentially. Implement try-except blocks for robust error handling during API failures.

Milestone 4: Frontend Development

- **Objective:** To build a responsive, intuitive, and aesthetic user interface.
- **Activity 4.1 - UI Design:** Design `index.html` with form elements capturing the Story Prompt, Character Name, Setting, Tone, and Art Style.

- **Activity 4.2 - Jinja2 Templating:** Develop `comic_preview.html` to dynamically loop through the backend layout dictionary, rendering panel titles, AI images, and narrations cleanly on the screen.

Milestone 5: Deployment and Testing

- **Objective:** To run the application locally, test all functionalities, and ensure deployment readiness.
- **Activity 5.1 - Local Deployment:** Configure environment variables (`.env`) for `GEMINI_API_KEY` and `HF_API_KEY` securely. Start the Uvicorn server to host the FastAPI application on localhost.
- **Activity 5.2 - System Testing:** Execute end-to-end testing by submitting various prompts. Verify that the AI correctly interprets the tone and art style, the images are generated accurately, and the PDF exports flawlessly without formatting errors.

5. RISK MANAGEMENT STRATEGY

During the planning phase, potential risks involving external APIs were identified, and mitigation strategies were developed:

- **Risk 1: External API Rate Limits (HTTP 402/500/503 Errors).**
 - *Mitigation:* Implement robust try-except blocks. If an API times out or blocks an IP, the system uses a fallback mechanism (e.g., dual-engine API switching) to ensure continuous operation.
- **Risk 2: DNS Resolution Errors (Errno 11001).**
 - *Mitigation:* Integrate proxy handlers (`urllib.request.ProxyHandler`) to bypass local network ISP blocks or DNS resolution failures when calling the Hugging Face API.
- **Risk 3: Total API Failure.**
 - *Mitigation:* Programmed a final safety net that dynamically generates a graphical 'Comic Caption Panel' (using Python PIL) if all external image APIs fail, ensuring the application never crashes and the PDF always generates successfully.

6. RESOURCE ALLOCATION

- **Frontend Design:** Focus on HTML, CSS, and Jinja2 dynamic rendering.
- **Backend Architecture:** Focus on FastAPI routing, Pydantic schema validation, and PDF compilation.
- **AI & Integration:** Focus on prompt engineering, API integrations (Gemini & Diffusers), and handling JSON responses accurately.
- **Quality Assurance:** Focus on network testing, error handling, and verifying the PDF structural integrity.